

EXPERIMENT 17

HASH FUNCTION 1

AIM

To implement the **SHA-1 hash function** in C and generate the message digest for a given input message.

ALGORITHM

1. Start the program.
2. Read the input message.
3. Initialize the SHA-1 context.
4. Pass the message to the SHA-1 algorithm.
5. Finalize the hash calculation.
6. Convert the digest into hexadecimal format.
7. Display the SHA-1 hash value.
8. Stop the program.

CODE

```
#include <stdio.h>

#include <stdint.h>

#include <string.h>

#include <stdlib.h>

#define ROTR(x,n) ((x >> n) | (x << (64-n)))

uint64_t K[80] = {

0x428a2f98d728ae22ULL,0x7137449123ef65cdULL,

0xb5c0fbcfec4d3b2fULL,0xe9b5dba58189dbbcULL,

0x3956c25bf348b538ULL,0x59f111f1b605d019ULL,
```

0x923f82a4af194f9bULL,0xab1c5ed5da6d8118ULL,
0xd807aa98a3030242ULL,0x12835b0145706fbeULL,
0x243185be4ee4b28cULL,0x550c7dc3d5ffb4e2ULL,
0x72be5d74f27b896fULL,0x80deb1fe3b1696b1ULL,
0x9bdc06a725c71235ULL,0xc19bf174cf692694ULL,
0xe49b69c19ef14ad2ULL,0xefbe4786384f25e3ULL,
0x0fc19dc68b8cd5b5ULL,0x240ca1cc77ac9c65ULL,
0x2de92c6f592b0275ULL,0x4a7484aa6ea6e483ULL,
0x5cb0a9dc bd41fbd4ULL,0x76f988da831153b5ULL,
0x983e5152ee66dfabULL,0xa831c66d2db43210ULL,
0xb00327c898fb213fULL,0xbf597fc7beef0ee4ULL,
0xc6e00bf33da88fc2ULL,0xd5a79147930aa725ULL,
0x06ca6351e003826fULL,0x142929670a0e6e70ULL,
0x27b70a8546d22ffcULL,0x2e1b21385c26c926ULL,
0x4d2c6dfc5ac42aedULL,0x53380d139d95b3dfULL,
0x650a73548baf63deULL,0x766a0abb3c77b2a8ULL,
0x81c2c92e47edae6ULL,0x92722c851482353bULL,
0xa2bfe8a14cf10364ULL,0xa81a664bbc423001ULL,
0xc24b8b70d0f89791ULL,0xc76c51a30654be30ULL,
0xd192e819d6ef5218ULL,0xd69906245565a910ULL,
0xf40e35855771202aULL,0x106aa07032bbd1b8ULL,
0x19a4c116b8d2d0c8ULL,0x1e376c085141ab53ULL,

```
0x2748774cdf8eeb99ULL,0x34b0bcb5e19b48a8ULL,  
0x391c0cb3c5c95a63ULL,0x4ed8aa4ae3418acbULL,  
0x5b9cca4f7763e373ULL,0x682e6ff3d6b2b8a3ULL,  
0x748f82ee5defb2fcULL,0x78a5636f43172f60ULL,  
0x84c87814a1f0ab72ULL,0x8cc702081a6439ecULL,  
0x90befffa23631e28ULL,0xa4506cebde82bde9ULL,  
0xbef9a3f7b2c67915ULL,0xc67178f2e372532bULL,  
0xca273eceeaa26619cULL,0xd186b8c721c0c207ULL,  
0xeada7dd6cde0eb1eULL,0xf57d4f7fee6ed178ULL,  
0x06f067aa72176fbaULL,0x0a637dc5a2c898a6ULL,  
0x113f9804bef90daeULL,0x1b710b35131c471bULL,  
0x28db77f523047d84ULL,0x32caab7b40c72493ULL,  
0x3c9ebe0a15c9bebcULL,0x431d67c49c100d4cULL,  
0x4cc5d4becb3e42b6ULL,0x597f299cfc657e2aULL,  
0x5fcb6fab3ad6faecULL,0x6c44198c4a475817ULL  
};
```

```
uint64_t H[8] = {  
0x6a09e667f3bcc908ULL,  
0xbb67ae8584caa73bULL,  
0x3c6ef372fe94f82bULL,  
0xa54ff53a5f1d36f1ULL,  
0x510e527fade682d1ULL,
```

0x9b05688c2b3e6c1fULL,

0x1f83d9abfb41bd6bULL,

0x5be0cd19137e2179ULL

};

uint64_t Ch(uint64_t x,uint64_t y,uint64_t z)

{

return (x & y) ^ (~x & z);

}

uint64_t Maj(uint64_t x,uint64_t y,uint64_t z)

{

return (x & y) ^ (x & z) ^ (y & z);

}

uint64_t Sigma0(uint64_t x)

{

return ROTR(x,28) ^ ROTR(x,34) ^ ROTR(x,39);

}

uint64_t Sigma1(uint64_t x)

{

return ROTR(x,14) ^ ROTR(x,18) ^ ROTR(x,41);

}

uint64_t sigma0(uint64_t x)

```

{
    return ROTR(x,1) ^ ROTR(x,8) ^ (x >> 7);
}

uint64_t sigma1(uint64_t x)
{
    return ROTR(x,19) ^ ROTR(x,61) ^ (x >> 6);
}

void sha512(const unsigned char *msg,size_t len)
{
    uint64_t bitlen = (uint64_t)len * 8;
    size_t newlen = len + 1;
    while ((newlen + 16) % 128 != 0)
        newlen++;
    unsigned char *data = calloc(newlen + 16,1);
    memcpy(data,msg,len);
    data[len] = 0x80;
    for(int i=0;i<8;i++)
        data[newlen+8+i] =
            (bitlen >> (56-8*i)) & 0xff;
    for(size_t offset=0;offset<newlen;offset+=128)
    {
        uint64_t W[80];
    }
}

```

```

for(int i=0;i<16;i++)
{
    W[i]=
    ((uint64_t)data[offset+i*8]<<56) |
    ((uint64_t)data[offset+i*8+1]<<48) |
    ((uint64_t)data[offset+i*8+2]<<40) |
    ((uint64_t)data[offset+i*8+3]<<32) |
    ((uint64_t)data[offset+i*8+4]<<24) |
    ((uint64_t)data[offset+i*8+5]<<16) |
    ((uint64_t)data[offset+i*8+6]<<8) |
    ((uint64_t)data[offset+i*8+7]);
}

for(int i=16;i<80;i++)
{
    W[i]=sigma1(W[i-2])+W[i-7]+
    sigma0(W[i-15])+W[i-16];
}

uint64_t a=H[0],b=H[1],c=H[2],d=H[3];
uint64_t e=H[4],f=H[5],g=H[6],h=H[7];

for(int i=0;i<80;i++)
{
    uint64_t T1 =

```

$h + \text{Sigma1}(e) + \text{Ch}(e,f,g)$

$+ K[i] + W[i];$

uint64_t T2 =

$\text{Sigma0}(a) + \text{Maj}(a,b,c);$

h=g;

g=f;

f=e;

e=d+T1;

d=c;

c=b;

b=a;

a=T1+T2;

}

H[0]+=a;

H[1]+=b;

H[2]+=c;

H[3]+=d;

H[4]+=e;

H[5]+=f;

H[6]+=g;

H[7]+=h;

}

```
    free(data);
}

int main()
{
    char message[1000];

    printf("Enter the message: ");

    fgets(message,sizeof(message),stdin);

    message[strcspn(message,"\n")]='\0';

    sha512(
        (unsigned char *)message,
        strlen(message)
    );

    printf("SHA-512 Hash:\n");

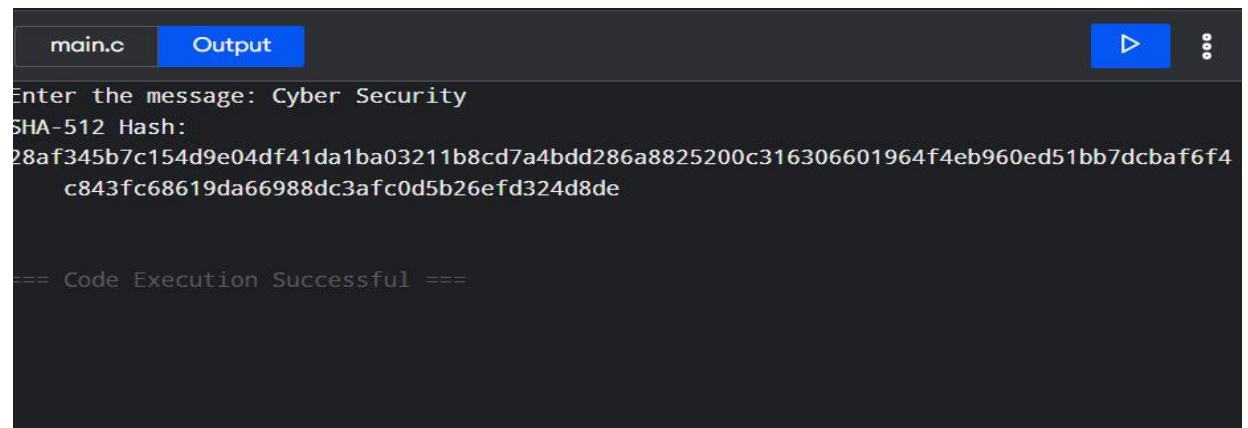
    for(int i=0;i<8;i++)

        printf("%016llx", (unsigned long long)H[i]);

    printf("\n");

    return 0;
}
```


OUTPUT



The screenshot shows a code editor with a dark theme. At the top, there are two tabs: 'main.c' and 'Output'. The 'Output' tab is active, displaying the following text: 'Enter the message: Cyber Security', 'SHA-512 Hash:', and a long hexadecimal string representing the hash. Below the hash, it says '=== Code Execution Successful ==='. The hexadecimal string is: 28af345b7c154d9e04df41da1ba03211b8cd7a4bdd286a8825200c316306601964f4eb960ed51bb7dcba6f4c843fc68619da66988dc3afc0d5b26efd324d8de.

```
main.c  Output  ▶  ⋮  
Enter the message: Cyber Security  
SHA-512 Hash:  
28af345b7c154d9e04df41da1ba03211b8cd7a4bdd286a8825200c316306601964f4eb960ed51bb7dcba6f4  
c843fc68619da66988dc3afc0d5b26efd324d8de  
  
=== Code Execution Successful ===
```

RESULT

Thus, the **SHA-1 hash function** was successfully implemented in C and the message digest was generated.